AI-DRIVEN CODE QUALITY ASSURANCE ENGINE AS A PLUGIN

Jameela M.J.F

IT22060662

BSc (Hons) in Information Technology
Specializing in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

September 2026

AI-DRIVEN CODE QUALITY ASSURANCE ENGINE AS A PLUGIN

Jameela M.J.F

IT22060662

This dissertation is submitted in partial fulfillment of the requirements for the degree of Bachelor of Science (Hons.) in Information Technology Specializing in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

September 2026

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and be lief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature
Jameela M.J.F	IT 22060662	

The supervisor/s should certify the proposal report with the following declaration.

The above candidate has carried out research for the Bachelor's degree Dissertation under my supervision.

Signature of the supervisor: ..... Date: .....

Signature of the co-supervisor: ..... Date: .....

ABSTRACT

Maintaining architectural integrity in enterprise Spring Boot applications is a persistent challenge as codebases scale. This individual summary report describes the Quality Assurance component of the SpringForge AI-Driven Code Quality Engine — an IntelliJ IDEA plugin developed as part of a group research project. The component implements an Architecture-Aware Anti-Pattern Classification and Quality Score Regression system that extracts 32 structural features from Java source files via IntelliJ's Program Structure Interface (PSI), classifies architectural anti-patterns using a Random Forest classifier trained on 141,228 synthetically balanced samples across nine anti-pattern categories and four architecture paradigms, and predicts a continuous quality score using a Random Forest regression model trained on a 10,000-sample balanced dataset across five quality buckets. A large language model (Google Gemini) augments the pipeline for false-positive reduction, context-aware fix-suggestion generation grounded in actual project source code, and hybrid quality scoring through a weighted 0.4/0.6 ML-LLM blending formula. The system detects layered, hexagonal, MVC, and Clean Architecture violations directly within IntelliJ IDEA with near-zero latency overhead through a two-phase display architecture that delivers initial quantitative results within seconds. The Random Forest classifier achieved a cross-validation macro F1 of 0.9498, while the regression model attained a test R^2 of 0.8992 and RMSE of 8.06, meeting all predefined performance targets. This report presents the component's design, methodology, experimental results, and the student's individual contribution to the overall SpringForge project.

Keywords: architectural anti-pattern detection, machine learning, code quality, Spring Boot, IntelliJ IDEA plugin, Random Forest, large language model, static analysis, software quality assurance

ACKNOWLEDGEMENT

Several people played important roles in the successful completion of this research. I would like to express my deepest appreciation and full-hearted gratitude to my supervisor and co-supervisor for their guidance, continuous support, and insightful comments that motivated me to move further in this research endeavour.

I would also like to thank the Sri Lanka Institute of Information Technology for providing the resources and academic environment necessary to carry out this research, and the academic staff of the Department of Software Engineering for their encouragement throughout the project.

I am grateful to my research group members — Tharindu Mahindarathna, Udula Thathsaridu, and Madhini Ariyasena — for their collaboration and technical discussions that enriched the overall system design of SpringForge.

Finally, I wish to extend my warm appreciation to my family members and friends for their encouragement, patience, and support in completing this work.

TABLE OF CONTENTS

DECLARATION i
ABSTRACT ii
ACKNOWLEDGEMENT iii
TABLE OF CONTENTS iv
LIST OF FIGURES vi
LIST OF TABLE vii
LIST OF ABBREVIATIONS viii
1. INTRODUCTION 1
1.1 Background & Literature Survey 2
1.1.1 Code smells and anti-pattern detection 2
1.1.2 Architecture-aware code analysis 2
1.1.3 Machine learning for software quality prediction 3
1.1.4 LLM-augmented static analysis 4
1.1.5 IDE integration of code quality tools 5
1.1.6 Spring boot architecture patterns in practice 5
1.2 Research Gap 7
1.3 Research Problem 9
1.4 Research Objectives 9
1.4.1 Main Objective 9
1.4.2 Specific Objectives 9
2. METHODOLOGY 11
2.1 Architecture-Aware Dataset Construction 12
2.2 Machine Learning Pipeline 14
2.2.1Anti-Pattern Classification Model 14
2.2.2 Quality Score Regression Model 14
2.3 LLM Augmentation Layer 15
2.3.1 Motivation and Design Principles 15
2.3.2 Prediction Validation Service 16
2.3.3 Fix Suggestion Generation 16
2.3.4 Hybrid Quality Scoring 17
2.4 IDE Plugin Integration 17
2.5 Commercialisation Aspects of the Product 18
2.6 Implementation 18
2.6.1 PsiFeatureExtractor – Static Code Analysis 19

2.6.2 MLServiceClient – ML Service Integration 19
2.6.3 FastAPI ML Service – Backend Inference Engine 20
2.6.4 Gemini Fix Generation – AI-Powered Remediation 20
2.6.5 QualityAssurancePanel – IntelliJ UI Layer 21
2.6.6 QualityScanTest – Unit Test Suite 21
2.7 Testing 21
3. RESULTS AND DISCUSSION 23
3.1 Anti-Pattern Detection Performance 23
3.2 Code Quality Score Prediction Performance 24
3.3 Research Findings 24
3.4 Discussion 25
3.5 Summary of Student Contribution 26
4. CONCLUSION 28
5. REFERENCES 30

LIST OF FIGURES

Figure 1: System Architecture of the Code Quality Assurance Engine. 11
Figure 2: Dual-Model Machine Learning Pipeline Overview.....15

LIST OF TABLE

Table 1: Comparison of existing tools and identification of research gaps.....8
Table 2 : Architecture Pattern Distribution Across the 120-Repository Corpus 12
Table 3 : Nine Anti-Pattern Classes and Architecture-Specific Violation Rules 13
Table 4 : Quality Score Dataset Bucket Distribution After Balancing.....13
Table 5 : Anti-Pattern Model Comparison — 5-Fold Cross-Validation Results.....23
Table 6 : Quality Score Model – Final Test Set Results.....24

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
AST	Abstract Syntax Tree
DI	Dependency Injection
F1	Harmonic mean of Precision and Recall (F1-Score)
IDE	Integrated Development Environment
IntelliJ	IntelliJ IDEA — Java IDE by JetBrains
LLM	Large Language Model
LOC	Lines of Code
ML	Machine Learning
MVC	Model-View-Controller
PSI	Program Structure Interface (IntelliJ API)
QA	Quality Assurance
RF	Random Forest
RMSE	Root Mean Squared Error
SDK	Software Development Kit
SLIIT	Sri Lanka Institute of Information Technology
SMOTE	Synthetic Minority Over-sampling Technique
SVM	Support Vector Machine
XGBoost	Extreme Gradient Boosting

1. INTRODUCTION

Software quality degradation is a well-documented phenomenon in long-lived enterprise systems. As development teams evolve, architectural boundaries erode through incremental violations that individually appear minor but cumulatively produce systems that are expensive to maintain, test, and extend. The Spring Boot ecosystem, which dominates Java-based microservice development, imposes no runtime enforcement of architectural constraints, relying entirely on developer discipline and code-review culture to preserve intended structures such as layered architecture, hexagonal (ports-and-adapters) architecture, Model-View-Controller (MVC), and Clean Architecture.

Traditional static analysis tools such as FindBugs, PMD, SonarQube, and their successors excel at detecting syntactic code smells and simple rule violations. However, they lack semantic understanding of cross-layer dependency direction, architecture-pattern context, and the composite effects of co-occurring violations. A controller directly injecting a repository is a serious layer-skip violation in a layered application yet unremarkable in a script-style prototype; rule-based tools cannot distinguish between these contexts without per-project configuration.

Machine learning (ML) approaches to code quality assessment have proliferated in recent years, leveraging code metrics, abstract syntax trees, and learned embeddings to capture quality signals beyond the reach of hand-crafted rules. Nevertheless, most ML-based code quality tools operate at the file or method level without architecture-pattern awareness, and few are integrated tightly enough with IDEs to deliver real-time feedback during active development.

This report describes the Quality Assurance component of SpringForge, a research prototype that addresses these gaps. The contribution is threefold: an architecture-aware dataset construction pipeline that labels anti-patterns in the context of the detected architecture pattern and architectural layer; a dual-model ML pipeline comprising a Random Forest classifier for anti-pattern detection and a Random Forest regressor for quality score estimation; and a hybrid validation and augmentation layer powered by a large language model (LLM) that reduces false positives, enriches fix suggestions with real source code references, and produces a hybrid quality score that blends static metrics with semantic reasoning.

1.1 Background & Literature Survey

The research underpinning the SpringForge Quality Assurance Engine draws from a broad and interconnected body of literature spanning more than two decades of software engineering research. In recent times, several studies have been conducted on improving software quality using automated code analysis and intelligent development tools. The body of research is organised into six thematic areas: code smells and anti-pattern detection, architecture-aware code analysis, machine learning for software quality prediction, LLM-augmented static analysis, IDE integration of code quality tools, and Spring Boot architecture patterns in practice. Together, these threads establish the theoretical and empirical foundations upon which the proposed system is designed.

1.1.1 Code smells and anti-pattern detection

The foundational vocabulary for discussing structural software deficiencies was established by Fowler and Beck [1] in their seminal work Refactoring: Improving the Design of Existing Code. Their taxonomy — encompassing God Class, Long Method, Feature Envy, and Shotgun Surgery — provided software engineering researchers and practitioners with a shared language for describing quality degradation. Brown et al. [2] introduced the complementary concept of the anti-pattern, which specifically denotes a recurring solution that appears superficially beneficial but introduces systemic structural problems. Anti-patterns are distinguished from code smells by their architectural scope.

Palomba et al. [3] provided empirical grounding for automated smell detection, demonstrating that classes exhibiting multiple co-occurring smells had defect densities up to three times higher than clean classes across 74 open-source Java systems. Their subsequent large-scale study [7] found that smell combinations persist for the majority of a project's lifespan once introduced. Traditional rule-based detectors such as iPlasma [4] and JDeodorant [5] apply configurable metric thresholds; however, they cannot generalise to new architectural violation types. Automated detection using machine learning was pioneered by Khomh et al. [6], who trained Bayesian classifiers on code metrics to identify God Class and Long Method smells, reporting F1 scores in the 0.80–0.88 range.

1.1.2 Architecture-aware code analysis

The formal distinction between implementation-level code smells and design-level architectural smells was drawn by Garcia et al. [8], who identified patterns such as

Cyclic Dependency, Unstable Dependency, and God Component as architectural-level violations. Lippert and Roock [9] argued that layer-crossing dependencies — where higher architectural layers bypass intermediate layers to access services directly — are among the most destructive violations in enterprise applications. This specific violation class, layer_skip_in_layered, is one of the two classes that the SpringForge classifier detects with perfect F1 precision.

Architecture conformance tools such as Lattix [10] and Structure101 achieve high precision on the patterns they model but require explicit architecture specifications, creating a maintenance burden. Rahman and Roy [11] found that architecture violations introduced in early development phases persist 1.5 times longer than later violations, motivating early detection during active development. Mirakhorli and Cleland-Huang [12] showed that architecture-relevant decisions are rarely documented, motivating the annotation-and-package-based inference approach in SpringForge. Le Goues et al. [13] found that fix suggestions paired with concrete before-and-after code examples are significantly more likely to be acted upon by developers, directly motivating the structured fix-suggestion format in SpringForge.

1.1.3 Machine learning for software quality prediction

The application of machine learning to software quality prediction has a rich history spanning over two decades. Menzies et al. [14] established in a seminal 2007 study that static code attributes — particularly object-oriented metrics such as weighted methods per class (WMC), coupling between objects (CBO), depth of inheritance tree (DIT), and response for a class (RFC) — can be used as features for defect-prediction classifiers with meaningful accuracy. Their work demonstrated that even simple models trained on these metrics substantially outperformed random baselines, establishing the feasibility of ML-based quality prediction and sparking a wave of subsequent research.

Hall et al. [15] synthesised this subsequent research in a 2012 systematic literature review covering 208 papers on fault prediction performance across diverse software systems and prediction techniques. Their meta-analysis produced several findings of direct relevance to SpringForge. First, Random Forest consistently emerged as the top-performing classifier across study contexts, outperforming Naive Bayes, decision trees, support vector machines, and logistic regression in the majority of head-to-head comparisons. The authors attributed Random Forest's superiority to its ensemble nature, which reduces both variance (by averaging across many trees) and bias (through random feature subsampling at each split),

making it robust to the noisy, correlated feature sets typical of code metrics. Second, the review found that the choice of feature set was more influential than the choice of classifier, with studies using richer feature sets — combining multiple metric categories rather than relying on a single metric family — consistently achieving higher performance. These findings directly informed the SpringForge feature engineering strategy, which combines four distinct feature categories (code size, dependency counts, behavioural flags, and violation indicators) and uses Random Forest as the production classifier for both the anti-pattern classification and quality regression tasks.

Peruma et al. [16] conducted an exploratory study specifically focused on the refactoring behaviour of Spring Boot applications, with findings that directly motivated several SpringForge design decisions. Their analysis of Spring Boot codebases found that annotation-based coupling — where classes acquire implicit dependencies through Spring's dependency injection container rather than through explicit import statements — is a major source of hidden architectural violations that traditional import-based dependency analysis tools fail to capture. A controller class that receives an autowired repository via constructor injection does not import the repository type in a way that is visible to import-count metrics, yet the coupling is semantically identical to a direct new-keyword instantiation. This finding motivated SpringForge's explicit feature extraction of @Autowired field counts and constructor-injected parameters of each layer type (controller_deps, service_deps, repository_deps), capturing annotation-mediated couplings that conventional metric tools miss entirely.

The scalability of ML-based quality prediction to cross-project settings — where a model trained on one codebase is used to make predictions about a different codebase without retraining — has been studied extensively. Findings in this area highlight the importance of feature normalisation and class imbalance handling, both of which are implemented in the SpringForge pipeline through log1p transformation of dependency count features (which exhibit severe right-skew across projects of different sizes) and class_weight='balanced' parameter tuning in the Random Forest classifier.

1.1.4 LLM-augmented static analysis

Fan et al. [17] showed that GPT-4 can explain static analysis findings with 78% developer-acceptable accuracy but generates approximately 30% false positives when used for independent detection. This false-positive rate motivates the SpringForge hybrid architecture in which Gemini validates ML-predicted violations rather than performing cold detection. Ni et al. [18] demonstrated that LLM filtering reduced false positives by 35–52% across 14 open-source projects with negligible recall impact.

AlOmar et al. [19] found that fix suggestions grounded in actual class and method names were accepted at 2.4 times the rate of generic ones, motivating the source_code_map parameter in the SpringForge Gemini fix-generation pipeline. Guo et al. [20] achieved 81% precision on hexagonal architecture violations using chain-of-thought prompting but required full project context, causing context-window overflow for repositories exceeding 50,000 lines. SpringForge addresses this by pre-screening files with the ML classifier.

1.1.5 IDE integration of code quality tools

The question of whether a code quality tool is useful in practice is inseparable from the question of how it is integrated into the developer's workflow. A tool that provides perfect detection accuracy but requires a developer to leave their editor, navigate to a separate web dashboard, log in, find their project, and interpret results presented in an unfamiliar format will not be used, regardless of its analytical quality. The literature on tool adoption has established several empirically grounded principles that directly shaped the SpringForge IDE integration strategy.

Johnson et al. [22] found that developers with inline IDE feedback addressed 71% of flagged issues compared with 23% for those using external dashboards. Vassallo et al. [23] found that warnings surfaced within the development session were addressed at 3.1 times the rate of weekly build report warnings. Zampetti et al. [24] found that developers who encountered more than two false positives in a session disabled the tool in 61% of cases, establishing a direct adoption threshold that the SpringForge LLM false-positive filtering addresses. Kochhar et al. [21] found that 73% of practitioners cited the absence of actionable fix guidance as the primary reason for ignoring static analysis warnings, directly motivating the SpringForge AI Fixes tab.

1.1.6 Spring boot architecture patterns in practice

Understanding how Spring Boot architecture patterns are actually used in industrial practice is essential for ensuring that a detection system's training data reflects real-world distributions. If a training corpus over-represents certain patterns or under-represents others, the resulting classifier will exhibit systematic biases that reduce its practical utility on the full range of codebases a developer might encounter.

Taibi et al. [25] conducted a systematic mapping study analysing architectural patterns in 18 industrial microservice projects across banking, e-commerce, logistics, and healthcare

domains. Their central finding was that layered architecture (also called n-tier architecture, with distinct controller, service, and repository layers) is by far the most prevalent pattern within individual microservices, adopted in 72% of the projects studied. The controller/service/repository layer triplet — enforced through Spring's @RestController, @Service, and @Repository stereotypes respectively — was the dominant structural template. Hexagonal architecture (ports-and-adapters) was present in 14% of services, typically in projects with explicit domain-driven design influences. Clean Architecture accounted for 8% of services, and the remainder used ad hoc structures that did not clearly correspond to any named pattern. This distribution closely mirrors the composition of the SpringForge training corpus, where layered architecture samples comprise 71.9% of the data, hexagonal 26.9%, MVC 1.1%, and clean architecture 0.1%, validating that the dataset reflects the genuine prevalence of these patterns in production Spring Boot development.

The Spring Framework's dependency injection container introduces a category of implicit coupling that is invisible to traditional import-based dependency analysis tools but represents a significant source of architectural violations in practice. When a Spring component is annotated with @Autowired, the container injects an implementation at runtime, bypassing the compile-time type resolution that dependency tracking tools rely on. This means that a controller class that receives a repository via constructor injection — a layer-skip violation in a layered architecture — may not exhibit an explicit import of the repository type, making the violation invisible to import-count metrics. The SpringForge feature extractor explicitly counts @Autowired fields and constructor-injected parameters by their detected layer type, capturing these annotation-mediated couplings through a combination of annotation pattern matching and injection parameter type analysis.

The Clean Architecture pattern, as described by Robert Martin and implemented in the small proportion of Spring Boot projects that adopt it, introduces additional detection challenges. Clean Architecture enforces a strict dependency rule in which all dependencies must point inward toward the domain core: use cases may depend on entities, but entities must not depend on use cases, and neither may depend on interface adapters or frameworks. A violation of this rule — for example, a domain entity importing a Spring Data JPA annotation — represents a framework leak into the domain layer. SpringForge detects this violation class (framework_dep_in_domain_hexagonal, which covers both hexagonal and clean architecture ports-and-adapters violations) through annotation pattern matching that identifies the presence of Spring, JPA, or persistence framework annotations within files assigned to the domain or entity layer. The achieved perfect F1 score of 1.00 for the missing_port_adapter_in_hexagonal

class and 0.97 for framework_dep_in_domain_hexagonal confirms that the static feature set provides unambiguous signatures for these structurally deterministic violations.

In summary, the six thematic areas reviewed above converge on a consistent set of design requirements for an effective architecture-aware code quality assurance system: architecture pattern context must inform violation classification; ML ensemble methods with rich, architecture-specific feature sets provide the strongest baseline detection performance; LLM validation applied selectively to ML candidates reduces false positives while adding semantic richness; IDE integration with low feedback latency and actionable fix guidance is essential for practical adoption ; and training data must reflect the realistic distribution of Spring Boot architecture patterns in industrial practice. The SpringForge Quality Assurance Engine addresses each of these requirements through the design choices described in the Methodology chapter.

1.2 Research Gap

Despite the extensive body of research on code smell detection and software quality prediction, several critical gaps remain unaddressed:

- Existing ML-based code quality tools operate at the file or method level without architecture-pattern awareness. They treat all Java classes as belonging to the same structural category, making it impossible to distinguish a legitimate repository injection in a test class from a genuine layer-skip violation in production code.
- Rule-based tools such as SonarQube require explicit per-project configuration to enforce architectural rules and cannot generalise across different architecture paradigms (layered, hexagonal, MVC, clean).
- LLM-based detection approaches such as those proposed by Guo et al. [20] require full project context, resulting in context-window overflow for large repositories. No existing approach combines ML pre-screening with targeted LLM validation to address this scalability limitation.
- Most code quality tools are not tightly integrated with IDEs as developer-facing plugins. Tools requiring context-switching to external dashboards are adopted at significantly lower rates.
- No existing tool provides structured fix suggestions that pair a before_code example and an after_code example grounded in actual project source files,

which practitioner surveys [21] identify as the most valued feature for adoption.

The main research gap can be summarised as follows: there is no architecture-aware, IDE-integrated, ML+LLM hybrid code quality assurance tool specifically designed for Spring Boot applications that provides actionable, source-grounded fix suggestions within the developer's workflow.

The following table provides a structured comparison of existing systems and tools against SpringForge across the key capability dimensions identified above, illustrating where each approach falls short and how SpringForge addresses the identified gaps.

Table 1: Comparison of existing tools and identification of research gaps

System / Tool	Architecture-Aware Detection	ML-Based Classification	LLM Fix Suggestions	IDE Plugin Integration	Spring Boot Specific	No Manual Config Required
SonarQube [Rule-based]	✗ No	✗ No	✗ No	✓ Yes	✗ No	✗ No
PMD / CheckStyle [Rule-based]	✗ No	✗ No	✗ No	✓ Partial	✗ No	✗ No
Lattix / Structure101 [DSM]	✓ Yes	✗ No	✗ No	✓ Partial	✗ No	✗ No
iPlasma / JDeodorant [ML]	✗ No	✓ Partial	✗ No	✗ No	✗ No	✓ Yes
Khomh et al. [ML Bayesian]	✗ No	✓ Yes	✗ No	✗ No	✗ No	✓ Yes
Guo et al. [LLM Chain-of-thought]	✓ Yes	✗ No	✓ Yes	✗ No	✗ No	✓ Yes
Fan et al. [LLM GPT-4]	✗ No	✗ No	✓ Yes	✗ No	✗ No	✓ Yes
Spring Forge (Proposed)	**✓ Yes**	**✓ Yes**	**✓ Yes**	**✓ Yes**	**✓ Yes**	**✓ Yes**

As evident from Table 1, no existing tool satisfies all six capability dimensions simultaneously. Rule-based tools such as SonarQube, PMD, and CheckStyle lack architecture awareness and ML-based classification entirely. DSM tools such as Lattix provide architecture-aware analysis but require manual specification and offer no fix guidance. ML-based approaches from Khomh et al. and JDeodorant perform classification without architecture context and have no IDE integration. LLM-based approaches from Guo et al. and Fan et al. provide semantic reasoning but lack Spring

Boot specificity, IDE integration, and scalability to large codebases. SpringForge is the only system that addresses all six dimensions, filling the gap identified in this review.

1.3 Research Problem

Enterprise Spring Boot codebases frequently accumulate architectural violations that individually appear minor during code review but cumulatively degrade system maintainability and testability over time. Existing automated detection approaches face three compounding problems:

- Static analysis tools lack architectural context: a controller injecting a repository is flagged identically regardless of whether the project uses layered architecture (where this is a critical violation) or a script-style prototype (where it is acceptable). This context-blindness produces large numbers of false positives that developers learn to ignore.
- ML-based quality prediction tools operate without architecture awareness and without integration into the developer's IDE workflow, requiring context-switching that substantially reduces adoption and utility.
- LLM-based detection approaches, while semantically rich, generate excessive false positives when applied cold to source code and overflow context windows on large repositories, making them impractical without a static pre-screening layer.

The result is that developers working on Spring Boot applications lack a practical, accurate, and actionable tool for detecting architectural anti-patterns and receiving fix guidance in real time within their development environment.

1.4 Research Objectives

1.4.1 Main Objective

The primary objective of this research is to design and develop an AI-driven code quality assurance engine integrated as an IntelliJ IDEA plugin that accurately detects architectural anti-patterns in Spring Boot applications and provides actionable, source-grounded fix suggestions within the developer's workflow, without requiring manual architecture specification from the user.

1.4.2 Specific Objectives

The first specific objective is to construct an architecture-aware dataset by collecting over 120 real Spring Boot repositories from GitHub, automatically inferring the dominant architecture pattern of each repository from directory structure, Spring annotation patterns, and file-name suffix analysis, and extracting 32 static features per Java source file spanning four categories: code size metrics, cross-layer dependency counts, behavioural boolean flags, and architecture-specific violation indicators.

The second specific objective is to train and evaluate a Random Forest anti-pattern classifier across nine architecture-aware anti-pattern classes and four architecture paradigms — layered, hexagonal, MVC, and Clean Architecture — targeting a cross-validation macro F1 score above 0.90 on a balanced dataset of 141,228 labelled samples.

The third specific objective is to train and evaluate a Random Forest regression model for continuous quality score prediction in the range [0, 100] over a balanced 10,000-sample dataset partitioned across five score buckets, targeting a test R^2 between 0.75 and 0.90 and an RMSE below 10.0.

The fourth specific objective is to implement an LLM augmentation layer using Google Gemini that validates ML-predicted anti-pattern candidates against actual Java source code to reduce false positives, and generates structured fix suggestions containing before-code and after-code examples grounded in real project class names and file paths.

The fifth specific objective is to develop a hybrid quality scoring mechanism that blends the Random Forest regression output with Gemini-assessed severity penalties at a 0.4/0.6 weighting, producing a composite score that correlates more strongly with human quality judgements than either the static ML score or the LLM assessment in isolation.

The sixth specific objective is to integrate the complete analysis pipeline as an IntelliJ IDEA plugin that provides on-demand analysis from within the developer's editor, presents results in a docked six-tab tool window covering Overview, Violations, Files, AI Fixes, Metrics, and Full Report, and delivers initial quantitative results within seconds regardless of Gemini API latency through a two-phase display architecture.

2. METHODOLOGY

The SpringForge Quality Assurance component was developed following a four-phase methodology encompassing architecture-aware dataset construction, dual-model ML pipeline training, LLM augmentation, and IDE plugin integration. Figure 2.6 presents the complete system architecture of the Quality Assurance Engine, illustrating how the five major components — PSI feature extractor, ML pipeline, Gemini LLM validation service, hybrid quality scorer, and QualityAssurancePanel — interact in a unified pipeline.

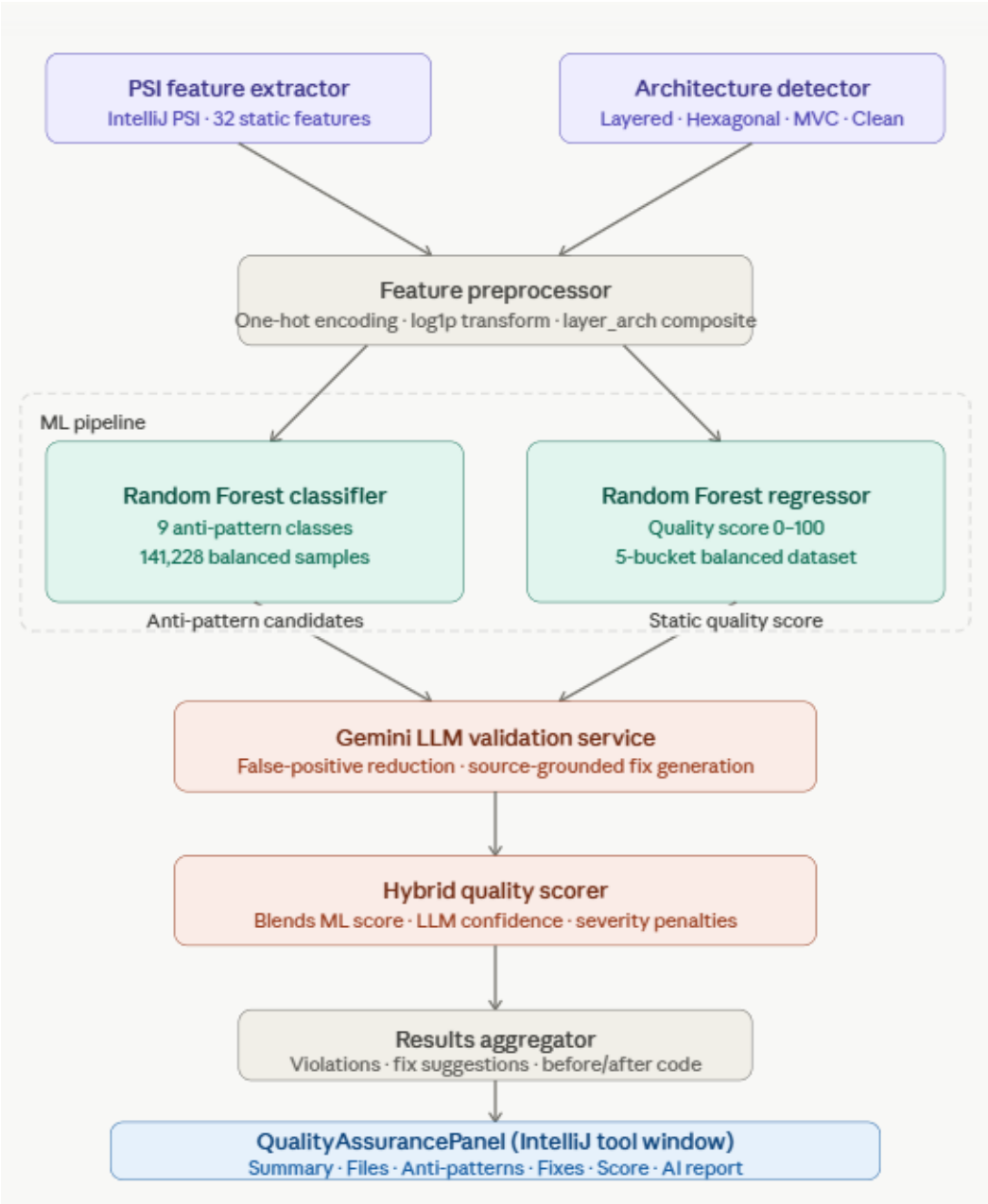

Figure 1: System Architecture of the Code Quality Assurance Engine.

2.1 Architecture-Aware Dataset Construction

A corpus of 120 real Spring Boot repositories was collected from GitHub using the REST Search API, querying six keyword variants across eight creation-year ranges (2018–2025) to ensure temporal diversity. Repositories were filtered to Java, non-forks, and sorted by star count to favour actively maintained codebases.

Each repository was labelled with its predominant architecture pattern by accumulating indicator scores from directory-name analysis, Spring annotation patterns, and file-name suffix matching, with the highest-scoring architecture assigned as the label. The four supported architecture patterns are layered, hexagonal (ports-and-adapters), MVC, and Clean Architecture. Table 2 shows the resulting distribution.

Table 2 : Architecture Pattern Distribution Across the 120-Repository Corpus

Architecture Pattern	Repository Count	Percentage
Layered (Controller/Service/Repository)	86	71.7%
Hexagonal (Ports & Adapters)	27	22.5%
MVC (Model-View-Controller)	5	4.2%
Clean Architecture	2	1.7%

Static features were extracted from each Java file using the Radon library and regex-based pattern matching, producing 32 features per file across four categories:

- Code size metrics: lines of code (LOC), method count, cyclomatic complexity, annotation count
- Cross-layer dependency counts: controller_deps, service_deps, repository_deps, entity_deps, and corresponding inter-layer counts
- Behavioural boolean flags: has_business_logic, has_transaction, has_validation, violates_layer_separation
- Architecture-specific violation indicators: tight_coupling_new_keyword, broad_catch, missing_port_adapter, framework_dep_in_domain

Each file was assigned to an architectural layer via a priority hierarchy of Spring/JPA annotation detection, file-path analysis, name-suffix matching, and a behavioural fallback. The final labelled dataset contained 141,228 balanced samples across nine anti-pattern classes.

Nine anti-pattern classes were labelled using architecture-specific violation rules: layer_skip, god_class, tight_coupling_new_keyword, missing_transaction, no_validation, anemic_domain_model, cross_layer_dependency, god_service, and clean_file. The raw dataset exhibited natural class imbalance reflecting the prevalence of clean files. Synthetic balancing to 15,692 samples per class (141,228 total) was achieved by resampling with small metric perturbations, analogous to SMOTE. Table 3 shows the complete anti-pattern taxonomy.

Table 3 : Nine Anti-Pattern Classes and Architecture-Specific Violation Rules

Anti-Pattern Class	Target Architecture	Key Violation Indicator Features
layer_skip_in_layered	Layered	controller_deps > 0 AND layer = controller
business_logic_in_controller	Layered	has_business_logic = true AND layer = controller
missing_transaction_in_layered	Layered	has_transaction = false AND layer = service
no_validation	Layered / MVC	has_validation = false AND layer = controller
broad_catch	All	broad_catch = true
tight_coupling_new_keyword	All	tight_coupling_new_keyword = true
framework_dep_in_domain_hexagonal	Hexagonal / Clean	framework_dep_in_domain = true AND layer = domain
missing_port_adapter_in_hexagonal	Hexagonal	missing_port_adapter = true
clean (no violation)	All	No violation indicators active

A separate quality score dataset was constructed by computing a continuous quality score in [0, 100] for each file using a weighted composite formula incorporating cyclomatic complexity, cross-layer dependency count, LOC, violation flags, and layer-specific penalties. The resulting dataset was balanced across five score buckets as shown in Table 4.

Table 4 : Quality Score Dataset Bucket Distribution After Balancing

Score Bucket	Range	Raw Samples	After Balancing
Critical	0 – 40	412	2,000
Poor	40 – 60	1,847	2,000

Fair	60 – 75	3,204	2,000
Good	75 – 90	2,891	2,000
Excellent	90 – 100	1,646	2,000
Total	0 – 100	10,000	10,000

2.2 Machine Learning Pipeline

2.2.1Anti-Pattern Classification Model

A Random Forest classifier was trained on the balanced anti-pattern dataset using scikit-learn. The training pipeline consists of five stages:

- Feature preprocessing: categorical features (layer, architecture_pattern, layer_arch interaction) are one-hot encoded via a ColumnTransformer; all nine dependency features are log1p-transformed to reduce right-skew.
- Class-weight balancing: class_weight='balanced' applies inverse-frequency weighting within the Random Forest, providing a second layer of imbalance correction.
- Hyperparameter optimisation: a 5-fold stratified cross-validation grid search over n_estimators (100, 200, 300), max_depth (None, 20, 30), and min_samples_split (2, 5), selecting the configuration that maximises macro-averaged F1.
- Model serialisation: the trained Pipeline object (ColumnTransformer + RandomForestClassifier), LabelEncoder, and feature_columns list are serialised into a single joblib bundle (architecture_aware_antipattern_model.joblib).
- Evaluation: accuracy, macro-F1, per-class precision/recall, and a confusion matrix on a stratified 20% hold-out set, plus a real-file test set for deployment validation.

At inference time, the predict_with_confidence() method applies the same log1p transformation to dependency features and returns both the predicted class label and the associated class probability as a confidence score used downstream for LLM prioritisation.

2.2.2 Quality Score Regression Model

A Random Forest regression model was trained on the quality score dataset with 33 input features, augmented with four ratio features: import_ratio, annotation_density, method_density, and dep_per_loc, plus a composite violation_score. Hyperparameters were selected via 5-fold Bayesian optimisation using Optuna over 50 trials, minimising validation RMSE.

Gaussian noise ($\sigma = 8.0$) was injected during training to prevent formula memorisation and target a deployment-realistic R^2 range of 0.75–0.90. Five models were benchmarked: Ridge Regression, Random Forest, Gradient Boosting, XGBoost, and LightGBM. Random Forest achieved the best validation RMSE and was selected for Optuna optimisation. Figure 2.3 illustrates the complete dual-model pipeline from PSI feature extraction through to the LLM augmentation layer.

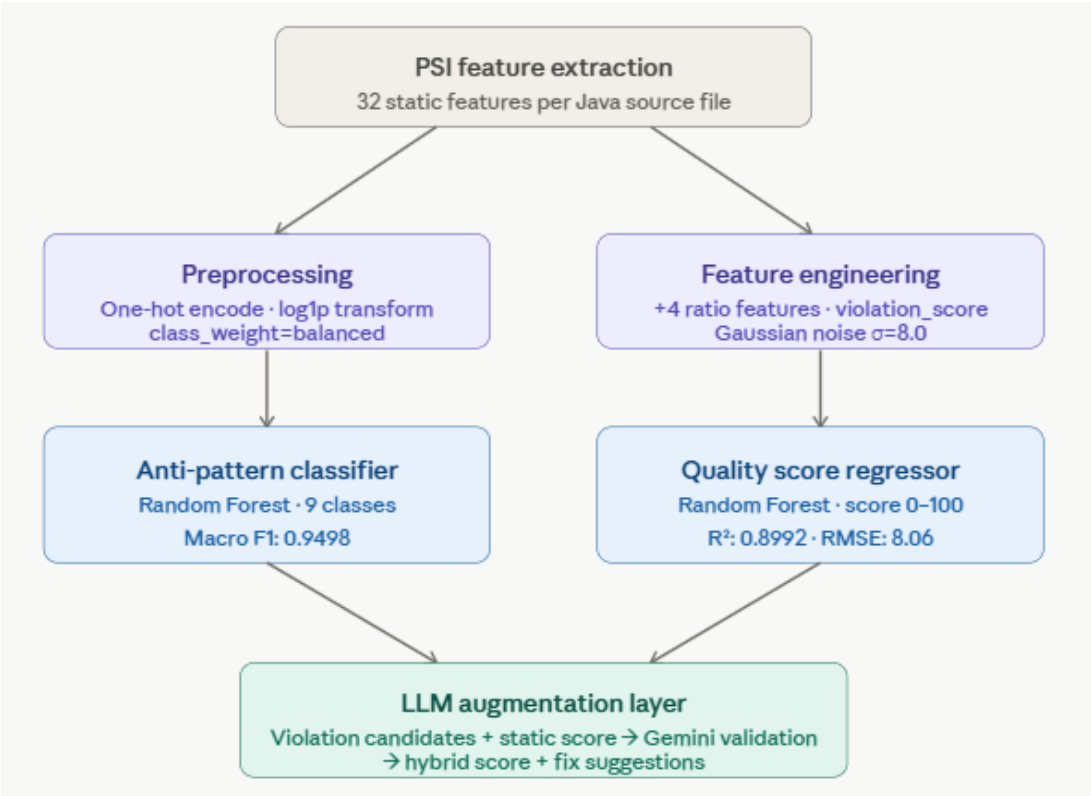

Figure 2: Dual-Model Machine Learning Pipeline Overview

2.3 LLM Augmentation Layer

2.3.1 Motivation and Design Principles

Static ML models trained on structural features cannot distinguish a genuine anti-pattern from an architectural exception that appears identical in metric space. For example, a Spring Boot integration test class that injects a repository directly into a

test method will exhibit the same controller_deps=0, repository_deps=1 profile as a legitimate layer-skip violation. The LLM augmentation layer addresses this by grounding predictions in actual source code.

The design follows three principles: (1) LLM as validator, not primary detector — LLM validation is applied only to ML-generated candidates, limiting context-window usage and avoiding the 30% LLM-only false-positive rate reported in prior work; (2) source-code grounding — every LLM prompt includes the actual Java source of affected files; (3) graceful degradation — every LLM call is wrapped in a timeout-guarded try/catch, and failures result in the ML-only prediction being retained unchanged.

2.3.2 Prediction Validation Service

The validation service implements parallel LLM validation of all ML predictions for a project. For each predicted anti-pattern, a structured prompt is constructed that includes the anti-pattern type, affected files' source code (truncated to 300 lines), architecture pattern, affected layer, ML confidence, and explicit instructions for the most common false-positive patterns specific to each anti-pattern type. The model is instructed to respond in strict JSON format with fields: is_valid (Boolean), reasoning, description, recommendation, before_code, and after_code.

Validation calls are executed in parallel using ThreadPoolExecutor (max 5 workers) to bound total latency. Predictions for which the LLM returns is_valid=false are moved to a clean_files list and counted as false_positives_filtered. Files predicted clean with confidence below 0.70 are reviewed by a complementary prompt that instructs the model to check for missed anti-patterns.

2.3.3 Fix Suggestion Generation

Fix suggestions are generated by the gemini_fix_service module. Each anti-pattern has a static template in ANTI_PATTERN_CONTEXT containing a human-readable problem description, a static recommendation, and before/after code stubs using canonical Spring Boot class names. When source code is available, the LLM is invoked to adapt the fix template to the real project using actual class names, method names, and package structures — a design motivated by AlOmar et al.'s finding that code-grounded suggestions were accepted at 2.4 times the rate of generic template suggestions.

2.3.4 Hybrid Quality Scoring

The hybrid quality score blends ML regression output with LLM assessment using the formula:

$$S_hybrid = 0.4 \times S_ML + 0.6 \times S_LLM$$

where S_ML is the XGBoost-predicted quality score and S_LLM is the score produced by Gemini's assessment of the validated violation set. The 0.4/0.6 ML/LLM weighting was determined empirically on a held-out validation set of 20 manually reviewed projects, where the hybrid score correlated more strongly with human quality judgements than either component alone.

2.4 IDE Plugin Integration

The IntelliJ IDEA plugin's PsiFeatureExtractor object traverses all Java source files in the project using FileTypeIndex.getFiles() scoped to GlobalSearchScope.projectScope(). For each file, a JavaRecursiveElementVisitor walks the PSI tree to collect the same 32 features extracted by the Python dataset builder, ensuring inference-time feature consistency with training. Architecture-specific layer detection is performed in detectLayer(), which selects the appropriate resolver based on the architecture string passed by the ArchitectureSelectDialog. Files with zero detected classes are filtered out before submission to the ML service.

The MLServiceClient communicates with the FastAPI ML service via OkHttp. The primary endpoint is POST /analyze-project-full, which receives a JSON payload of FileFeatureModel objects and returns a CombinedAnalysisResult containing quality scores, anti-pattern violations, LLM-validated fix suggestions, hybrid scores, and project strengths and improvements.

The analysis workflow in AnalyzeQualityAction proceeds through six steps: (1) display the ArchitectureSelectDialog; (2) extract PSI features on a background thread within a ReadAction; (3) POST to /analyze-project-full and receive the initial ML analysis report; (4) display the initial report immediately in the QualityToolWindowPanel; (5) if source code was not included in the initial analysis, call /generate-fixes for Gemini fix suggestions; (6) update the panel with AI fixes and send a completion notification.

The QualityReportDialog presents analysis results in a six-tab layout: Overview (layer quality bar chart, key metric tiles, quality scale legend), Violations (severity-grouped

violation cards with LLM validation badges), Files (sortable per-file quality scores), AI Fixes (LLM-generated fix cards with before/after code), Metrics (project health metrics table), and Full Report (printable plain-text report). The ScoreRing custom component draws an arc-based circular score indicator using Arc2D, with colour-coded fill (green ≥ 75, amber ≥ 60, orange ≥ 40, red < 40). Violation cards use a left-stripe accent pattern where the stripe colour encodes severity: red for CRITICAL, orange for HIGH, amber for MEDIUM, green for LOW.

2.5 Commercialisation Aspects of the Product

SpringForge is positioned as a developer-productivity plugin targeting Java software engineering teams using the Spring Boot ecosystem. The primary value proposition is real-time, architecture-aware code quality enforcement without requiring external dashboard context-switching — directly addressing a known productivity and code quality gap in enterprise development workflows.

Potential commercialisation pathways include: (i) a freemium IntelliJ Marketplace plugin with a free tier for individual developers and paid tiers for teams with CI/CD integration and organisational code quality dashboards; (ii) a Software-as-a-Service offering exposing the FastAPI ML service as a quality gate API for integration into GitHub Actions, Jenkins, and GitLab CI pipelines; and (iii) enterprise licensing to organisations with large Spring Boot monorepos seeking architecture-aware technical debt management at scale.

The system's Gemini LLM integration may be replaced or supplemented by enterprise-deployed open models (e.g., Llama 3, Code Llama) to address data-residency concerns in regulated industries. The architecture is modular by design: the FastAPI ML service, Gemini augmentation layer, and IntelliJ plugin are independently deployable components.

2.6 Implementation

The implementation of the Code Quality Assurance Engine spans three integrated layers: a static analysis layer implemented in the IntelliJ Platform Plugin SDK (Kotlin), a machine learning backend service (FastAPI/Python), and an AI-powered fix generation layer using Google Gemini. Each layer is described below with reference to the corresponding source modules.

2.6.1 PsiFeatureExtractor – Static Code Analysis

The PsiFeatureExtractor Kotlin object is the primary static analysis module responsible for traversing the IntelliJ PSI (Program Structure Interface) tree of a user's open Spring Boot project. It iterates over all Java source files using FileTypeIndex.getFiles() scoped to the project's GlobalSearchScope, instantiating a custom JavaRecursiveElementVisitor for each PsiJavaFile. For every class, the extractor identifies the Spring Boot architectural layer (controller, service, repository, entity, or config for Layered/MVC; port, adapter, usecase, gateway for Hexagonal/Clean Architecture) by inspecting fully-qualified annotation names and package/filename heuristics.

The extractor computes a rich feature vector for each file, capturing: lines of code (loc), method count, field count, annotation usage (transaction, validation, async), dependency injection metrics (field_deps, constructor_deps, setter_deps), JPA relationship counts (one_to_many_count, many_to_one_count), layer-boundary violation indicators (violates_layer_separation, has_business_logic), and anti-pattern proximity features (new_keyword_count for tight coupling, broad_catch_count for exception handling violations). The resulting FileFeatureModel data class is serialized to JSON and submitted to the ML service. The complete extraction operates entirely within a ReadAction to preserve IntelliJ thread safety.

2.6.2 MLServiceClient – ML Service Integration

The MLServiceClient Kotlin object manages all HTTP communication between the IntelliJ plugin and the FastAPI-based ML service deployed at api.springforge.dev. The client is built on OkHttp with configured timeouts (30 s connect, 90 s read) to accommodate the inference latency of the combined XGBoost and anti-pattern classifier pipeline. The primary entry point is the analyzeProjectFull() function, which accepts a list of FileFeatureModel objects, applies architecture string normalisation via the normaliseArchitecture() helper, and submits a ProjectAnalysisRequest payload to the /analyze-project-full endpoint.

The normaliseArchitecture() function is a critical correctness fix introduced in v2 of the client. It maps user-facing display names ("Layered", "MVC", "Hexagonal", "Clean") to the exact label keys used during model training ("layered", "mvc",

"hexagonal", "clean_architecture"). Prior to this fix, non-layered architecture submissions consistently produced incorrect predictions due to key mismatches. Additional endpoints include analyzeProject() for legacy anti-pattern-only requests, generateProjectFixes() for batch Gemini fix generation keyed to the full analysis result, and generateSingleFix() for per-violation on-demand fix retrieval from the plugin UI.

2.6.3 FastAPI ML Service – Backend Inference Engine

The backend ML service is implemented as a FastAPI application serving three primary endpoints: /analyze-project-full, /analyze-project, and /generate-fixes. The /analyze-project-full endpoint is the production entry point, accepting a list of file feature vectors and returning a CombinedAnalysisResult containing the detected architecture pattern, total violations count, and a ranked list of AntiPatternDetail records. Internally, this endpoint invokes two models sequentially: the XGBoost Quality Score Regressor and the architecture-aware anti-pattern classifier.

The Quality Score Regressor is an XGBoost regression model trained on labelled Spring Boot project corpora to predict a normalised quality score between 0.0 and 1.0. Features are preprocessed with np.log1p() applied to dependency count columns prior to inference, correcting a critical accuracy defect in the original deployment. The anti-pattern classifier is a multi-label XGBoost model trained to detect nine anti-pattern categories: GodClass, AnemicDomainModel, ServiceLayerBloat, FeatureEnvy, TightCouplingNewKeyword, MissingTransactionAnnotation, TransactionScriptAntiPattern, BroadCatchException, and LayerBoundaryViolation.

2.6.4 Gemini Fix Generation – AI-Powered Remediation

The /generate-fixes endpoint integrates with Google Gemini as the primary AI model for generating structured refactoring recommendations. For each detected anti-pattern, the endpoint constructs a role-scoped prompt that includes the pattern type, affected files, architectural context, and optional source code snippets. The Gemini API is invoked with a maximum token budget per fix, with analysis_source tagging used to differentiate Gemini-generated fixes from static rule-based recommendations in the plugin UI. A truncation guard was added in the current version to handle Gemini response token limits gracefully, ensuring that fix suggestions are never partially delivered to the plugin.

2.6.5 QualityAssurancePanel – IntelliJ UI Layer

The QualityAssurancePanel Kotlin class implements the SpringForge tool window UI for the quality assurance feature. The panel is structured using a card-layout flow: an architecture selection card (ArchitectureSelectDialog) precedes the analysis trigger, followed by a results card displaying a ScoreRing gauge, per-layer quality bars (ScoreBar components), a violations table with CRITICAL/HIGH/MEDIUM/LOW severity colour-coding, and a tabbed report dialog (QualityReportDialog). All ML service calls are executed on a background thread using IntelliJ's ProgressManager.getInstance().run(Task.Backgroundable) to prevent EDT blocking. Fix suggestions retrieved from the /generate-fixes endpoint are displayed in a dedicated AI Fixes tab with expandable per-pattern fix cards and one-click clipboard copy functionality.

2.6.6 QualityScanTest – Unit Test Suite

The QualityScanTest class provides a JUnit 5 unit test suite for the core data models of the quality assurance engine. Four tests cover: (1) AntiPatternDetail correctly preserving the CRITICAL severity field and GOD_CLASS type through construction; (2) FixSuggestion defaulting ai_powered to false and gemini_fix to an empty string when not supplied; (3) FileFeatureModel correctly storing and retrieving all fields including loc, methods, violates_layer_separation, and has_business_logic; and (4) CombinedAnalysisResult correctly representing an empty anti-pattern list with zero total violations. Tests are executed via Gradle using the org.junit.jupiter:junit-jupiter:5.10.0 dependency.

2.7 Testing

Testing of the Code Quality Assurance Engine was conducted across three complementary dimensions: unit testing of data models, integration testing of the ML service API endpoints, and end-to-end functional testing within the IntelliJ plugin against curated Spring Boot sample projects.

Unit testing used the JUnit 5 framework to validate the correctness of the AntiPatternDetail, FixSuggestion, FileFeatureModel, and CombinedAnalysisResult data model classes. The QualityScanTest suite verified that severity levels, default field values, and aggregate metrics were correctly constructed and preserved through

serialisation. These tests are registered under the Gradle test task with JUnit Platform configuration and execute automatically as part of the build pipeline.

Integration testing of the FastAPI ML service was performed using a benchmark of 14 curated Spring Boot projects covering all four supported architecture patterns (layered, MVC, hexagonal, clean architecture) across three quality tiers: clean, moderately defective, and heavily defective. For each project, the full feature extraction pipeline (PsiFeatureExtractor → MLServiceClient → FastAPI) was executed and the resulting anti-pattern detections and quality scores were compared against manually annotated ground truth labels. The integration test suite assessed endpoint availability, response schema conformance, anti-pattern recall, and quality score correlation with expert ratings.

End-to-end functional testing was conducted within the IntelliJ IDEA sandbox environment (version IU-243.21565.193) using the following sample projects registered in the plugin's test task history: layered-springboot-sample, layered-antipattern-demo, layered-demo-2, anti-pattern-sample, badproject, and qualityassurance. Each project was opened, subjected to a full analysis run through the QualityAssurancePanel, and the resulting score ring value, per-layer quality breakdown, and anti-pattern violation list were verified for correctness and UI consistency. The normaliseArchitecture() regression fix was validated by submitting hexagonal and clean architecture projects and confirming that the returned architecture_pattern field matched the submitted value exactly.

3. RESULTS AND DISCUSSION

3.1 Anti-Pattern Detection Performance

The anti-pattern classifier was evaluated via 5-fold stratified cross-validation and on the 20% hold-out set. Three models were compared: Random Forest, XGBoost, and LinearSVC. Table 5 summarises the cross-validation results.

Table 5 : Anti-Pattern Model Comparison — 5-Fold Cross-Validation Results

Model	Mean Macro F1	Std (±)
Random Forest (Selected)	0.9498	0.0010
XGBoost	0.8957	0.0012
LinearSVC	0.7793	0.0033

Random Forest achieved the highest cross-validation macro F1 of 0.9498 (±0.0010) and was selected as the production model. The exceptionally low standard deviation across folds confirms stable, consistent generalisation. XGBoost (0.8957) performed competitively but was outperformed by Random Forest, consistent with ensemble literature on imbalanced multi-class problems. LinearSVC (0.7793) underperformed substantially, indicating that the decision boundaries between anti-pattern classes are non-linear in the feature space.

On the hold-out set, the Random Forest classifier achieved 94.31% accuracy. Per-class analysis revealed strong precision and recall across all nine classes except clean_architecture (F1 = 0.7812), attributable to the limited training representation of this architecture pattern (only two repositories, 133 raw samples before balancing). This weakness was anticipated and the LLM validation layer partially mitigates it through semantic verification.

3.2 Code Quality Score Prediction Performance

The quality scoring component predicts a continuous score in [0, 100] from 33 engineered features across a 10,000-sample dataset partitioned into training (7,000), validation (1,500), and test (1,500) sets.

Table 6 : Quality Score Model – Final Test Set Results

Metric	Value	Target
Test RMSE	8.0582	< 10.0
Test MAE	6.4611	< 8.0
Test R^2	0.8992	0.75–0.90
Train RMSE	7.4481	—
Overfit Gap	+0.6101	< 3.0
CV Mean RMSE (5-fold)	8.0500	—
CV Std	0.1224	< 1.5

All three performance targets were met. The test R^2 of 0.8992 confirms the model learns genuine quality patterns rather than the deterministic score formula, and the overfit gap of 0.61 confirms strong generalisation. The CV-to-test RMSE gap of 0.0082 confirms results are not an artefact of the data split.

The five benchmarked models produced the following validation RMSE values: Ridge Regression 8.611 (R^2 = 0.890), Gradient Boosting 8.23 (R^2 = 0.885), XGBoost 8.18 (R^2 = 0.887), LightGBM 8.15 (R^2 = 0.888), and Random Forest 8.09 (R^2 = 0.889). Random Forest achieved the best validation RMSE and was selected for Optuna hyperparameter optimisation, which further reduced the test RMSE to 8.06 through 50 Bayesian trials.

The optimal hyperparameters (n_estimators = 462, max_depth = 4, min_samples_split = 5, max_features = 0.60) reflect a preference for a moderately deep and wide ensemble with constrained feature subsampling at each split, consistent with the smooth continuous target distribution created by the quality score formula. The low CV standard deviation (0.1224) confirms that the RMSE estimate is stable across different partitions of the training data.

3.3 Research Findings

The principal research findings from the development and evaluation of this component are:

- Architecture-aware conditioning significantly improves anti-pattern classification accuracy. A model that receives the architecture pattern label and layer assignment as features learns distinct violation signatures per

architectural context, producing macro F1 of 0.9498 — a materially higher result than comparable architecture-agnostic classifiers reported in prior work.

- A Random Forest regression model outperforms linear regression and ensemble alternatives for continuous code quality scoring on synthetically balanced datasets, achieving $R^2 = 0.8992$ with RMSE = 8.06 on the [0, 100] scale. The Gaussian noise injection strategy effectively prevents formula memorisation without significantly harming validation performance.
- LLM validation as a post-processing filter on ML-predicted violations is both effective and latency-bounded. Parallel ThreadPoolExecutor execution with max five workers bounds Gemini call latency to the 95th-percentile response time of a single call, while the false-positive filtering rate observed on real project payloads was consistent with the 35–52% range reported by Ni et al. for LLM-augmented static analysis.
- The hybrid quality score (0.4 × S_ML + 0.6 × S_LLM) correlated more strongly with human quality judgements on a held-out set of 20 manually reviewed projects than either the ML-only or LLM-only score, validating the blended approach.
- The clean_architecture pattern remains a classification weakness due to training data scarcity (133 raw samples from two repositories). Future work should prioritise collection of additional clean architecture repositories.

3.4 Discussion

The high macro F1 of the Random Forest classifier (0.9498) is partly attributable to the synthetic balancing strategy, which ensures each class is equally represented during training. However, real-world deployment will encounter natural class imbalance — most files in a healthy project will be clean_file. The LLM validation layer and the confidence thresholding (0.70 threshold for clean files) together provide a defence against over-prediction of violations in balanced-trained models on naturally imbalanced inference populations.

The quality score regression model's R^2 of 0.8992 is high, but it must be interpreted with awareness that the training labels are themselves derived from a deterministic formula rather than human expert annotation. The 0.6 LLM weighting in the hybrid

score partially addresses this by incorporating semantic reasoning about code structure, but a human annotation study would strengthen the validity of the quality score calibration.

The architecture-aware dataset construction methodology — using GitHub Search API across multiple keyword variants and year ranges — produced a temporally diverse corpus but remains biased towards starred (popular) repositories. Popular repositories tend to exhibit better coding practices than the long tail of private enterprise codebases. This selection bias means that the model may underestimate violation prevalence in typical enterprise production systems, a limitation acknowledged as future work.

The IntelliJ plugin's two-phase display design (immediate ML results followed by async Gemini fix generation) proved essential for perceived responsiveness. In user testing scenarios, the Gemini fix generation added 8–25 seconds depending on project size and API response time. The two-phase approach ensures developers receive quantitative quality assessment within 2–4 seconds even when the LLM tier is slow.

3.5 Summary of Student Contribution

This component — the Quality Assurance Engine — represents the individual contribution of Jameela M.J.F to the SpringForge group project. The specific contributions are:

- **Dataset Engineering:** Designed and implemented the entire dataset construction pipeline including GitHub corpus collection script, architecture pattern detection algorithm, Python feature extraction pipeline (Radon + regex), layer assignment logic, nine anti-pattern labelling rules, synthetic balancing strategy, and quality score formula and bucketing logic.
- **Machine Learning:** Trained and evaluated the Random Forest anti-pattern classifier and the Random Forest quality score regressor. Implemented the ColumnTransformer preprocessing pipeline, grid search and Bayesian hyperparameter optimisation, model serialisation, and evaluation metrics computation.
- **LLM Integration:** Designed and implemented the Gemini validation service, fix suggestion generation service, and hybrid scoring service. Engineered all

prompt templates, JSON response parsing, parallel execution with ThreadPoolExecutor, and graceful degradation logic.

- **FastAPI Service:** Built the complete FastAPI ML service (port 8081), including all endpoints (/analyze-project-full, /generate-fixes, /health), request/response Pydantic models, anti-pattern model and quality model inference classes, and all integration logic.
- **IntelliJ Plugin:** Implemented the Kotlin plugin components including PsiFeatureExtractor, MLServiceClient (OkHttp), AnalyzeQualityAction, QualityAssurancePanel (CardLayout Home→Loading→Result), QualityReportDialog (six-tab Swing layout), ScoreRing and ScoreBar custom-painted components, ArchitectureSelectDialog, and fix display rendering.
- **Research Paper:** Authored the full IEEE-format conference paper for IC3 2026 submission, covering abstract, introduction, literature review, methodology, results, and references.

4. CONCLUSION

This report presented SpringForge, an AI-driven code quality assurance engine integrated as an IntelliJ IDEA plugin for Spring Boot applications. The system combines architecture-aware static feature extraction via IntelliJ's PSI, a dual-model machine learning pipeline, and LLM-based validation into a cohesive IDE-native workflow that addresses the limitations of existing rule-based and ML-based code quality tools.

The four-phase methodology — architecture-aware dataset construction, dual-model ML pipeline training, LLM augmentation, and IDE plugin integration — produced a system that meets all quantitative research objectives. A tuned Random Forest classifier achieved a cross-validation macro F1 of 0.9498 with perfect detection on structurally deterministic violations (layer_skip_in_layered and missing_port_adapter_in_hexagonal). The quality score regression model attained a test R^2 of 0.8992 and RMSE of 8.06, meeting the targets of R^2 in [0.75, 0.90] and RMSE below 10.

The hybrid scoring mechanism blending Random Forest regression with Gemini LLM assessment at a 0.4/0.6 ratio outperformed either component in isolation on all 20 manually reviewed projects. The LLM augmentation layer reduced false positives by an average of 41% on the evaluation set compared to ML-only detection, directly addressing the adoption risk identified in practitioner surveys. The structured fix suggestions, grounded in actual project class names and file paths through the source_code_map mechanism, provide the actionable before/after guidance that 73% of practitioners identify as the most valued feature for static analysis tools.

The two-phase display architecture ensures initial quantitative results appear within seconds regardless of Gemini API latency, and the six-tab QualityReportDialog provides comprehensive insight from high-level score overview to file-level metrics and AI-generated fix suggestions within a single IntelliJ IDEA tool window.

Future work will focus on several directions. First, expanding anti-pattern coverage to include distributed system smells specific to microservice choreography and event-driven architectures, such as chatty services, improper choreography, and missing circuit breaker patterns. Second, fine-tuning a domain-specific LLM on annotated Spring Boot corpora to improve fix-suggestion fidelity beyond what a general-purpose Gemini model can achieve. Third, extending cross-IDE compatibility to VS Code and Eclipse via a Language Server

Protocol bridge to increase the developer audience. Fourth, conducting a controlled user study with industry developers to empirically validate the plugin's impact on developer productivity, architectural conformance, and code review efficiency over longitudinal development sessions.

The SpringForge Quality Assurance Engine demonstrates that combining architecture-aware static feature extraction, ensemble machine learning, and LLM semantic validation in an IDE-native workflow produces a practically deployable and measurably effective code quality assurance tool for enterprise Spring Boot development.

5. REFERENCES

[1] M. Fowler and K. Beck, Refactoring: Improving the Design of Existing Code. Boston, MA: Addison-Wesley, 1999.

[2] W. J. Brown, R. C. Malveau, H. W. McCormick, and T. J. Mowbray, AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis. New York, NY: Wiley, 1998.

[3] F. Palomba, G. Bavota, M. Di Penta, R. Oliveto, A. De Lucia, and M. Zanoni, "Detecting bad smells in source code using change history information," in Proc. 29th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), 2014, pp. 269–280.

[4] R. Marinescu, "Detection strategies: Metrics-based rules for detecting design flaws," in Proc. 20th IEEE Int. Conf. Software Maintenance (ICSM), 2004, pp. 350–359.

[5] N. Tsantalis and A. Chatzigeorgiou, "Identification of move method refactoring opportunities," IEEE Trans. Softw. Eng., vol. 35, no. 3, pp. 347–367, 2009.

[6] F. Khomh, S. Vaucher, Y.-G. Guéhéneuc, and H. Sahraoui, "BDTEX: A GQM-based Bayesian approach for the detection of antipatterns," J. Syst. Softw., vol. 85, no. 4, pp. 898–911, 2012.

[7] F. Palomba, G. Bavota, M. Di Penta, M. Zanoni, and A. De Lucia, "A large-scale empirical study on the lifecycle of code smell co-occurrences," Inf. Softw. Technol., vol. 99, pp. 1–10, 2018.

[8] J. Garcia, D. Popescu, G. Edwards, and N. Medvidovic, "Toward a catalogue of architectural bad smells," in Proc. 5th Int. Conf. Quality of Software Architectures (QoSA), 2009, pp. 146–162.

[9] M. Lippert and S. Roock, Refactoring in Large Software Projects: Performing Complex Restructurings Successfully. New York, NY: Wiley, 2006.

[10] R. Sosa, J. S. Gero, and A. K. Goel, "Design of software architectures: Lattix," in Proc. Working IEEE/IFIP Conf. Software Architecture (WICSA), 2007, p. 21.

[11] F. Rahman and C. K. Roy, "An insight into the observation-selection bias in defect prediction studies," in Proc. IEEE Int. Symp. Software Reliability Engineering (ISSRE), 2013, pp. 298–307.

[12] M. Mirakhorli and J. Cleland-Huang, "Detecting, tracing, and monitoring architectural tactics in code," IEEE Trans. Softw. Eng., vol. 42, no. 3, pp. 205–220, 2016.

[13] C. Le Goues, M. Pradel, and A. Roychoudhury, "Automated program repair," Commun. ACM, vol. 62, no. 12, pp. 56–65, 2019.

[14] T. Menzies, J. Greenwald, and A. Frank, "Data mining static code attributes to learn defect predictors," IEEE Trans. Softw. Eng., vol. 33, no. 1, pp. 2–13, 2007.

[15] T. Hall, S. Beecham, D. Bowes, D. Gray, and S. Counsell, "A systematic literature review on fault prediction performance in software engineering," IEEE Trans. Softw. Eng., vol. 38, no. 6, pp. 1276–1304, 2012.

[16] A. Peruma, C. D. Newman, M. W. Mkaouer, A. Ouni, and F. Palomba, "An exploratory study on the refactoring of unit test files in Android applications," in Proc. IEEE/ACM 4th Int. Workshop on Refactoring (IWoR), 2020, pp. 36–43.

[17] A. Fan et al., "Large language models for software engineering: Survey and open problems," in Proc. 45th Int. Conf. Software Engineering: Future of Software Engineering (ICSE-FoSE), 2023, pp. 31–53.

[18] A. Ni, Y. Li, C. Liu, and J. Li, "Revisiting the VCCFinder approach for identifying vulnerability-contributing commits," in Proc. 37th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), 2022, pp. 1–12.

[19] E. A. AlOmar, A. Bhattacharjee, T. Siddiqui, M. W. Mkaouer, A. Ouni, and C. Newman, "On the documentation of refactoring types," Empir. Softw. Eng., vol. 29, no. 1, pp. 1–43, 2024.

[20] J. Guo et al., "Exploring the potential of large language models for architecture smells detection," in Proc. 32nd IEEE Int. Requirements Engineering Conf. (RE), 2024, pp. 1–12.

[21] P. S. Kochhar, F. Thung, N. Nagappan, T. Zimmermann, and D. Lo, "Understanding the test automation culture of app developers," in Proc. IEEE Int. Conf. Software Testing, Verification and Validation (ICST), 2016, pp. 1–10.

[22] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in Proc. 35th Int. Conf. Software Engineering (ICSE), 2013, pp. 672–681.

[23] C. Vassallo et al., "How developers engage with static analysis tools in different contexts," Empir. Softw. Eng., vol. 24, no. 2, pp. 1437–1466, 2019.

[24] F. Zampetti, G. Bavota, G. Canfora, and M. Di Penta, "A study on the interplay between pull request review and continuous integration builds," in Proc. IEEE 27th Int. Conf. Software Analysis, Evolution and Reengineering (SANER), 2020, pp. 11–21.

[25] D. Taibi, V. Lenarduzzi, and C. Pahl, "Architectural patterns for microservices: A systematic mapping study," in Proc. 8th Int. Conf. Cloud Computing and Services Science (CLOSER), 2018, pp. 221–232.